

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 11 · LECTURE NOTES

Multiple Agents and Self-Play

When the environment learns too.

MODULE OUTCOMES

- Explain non-stationarity and why it breaks the MDP assumption.
- Explain self-play and why it produces a curriculum.
- Explain the cycling problem and population-based responses.
- Distinguish cooperative from competitive multi-agent learning.
- Explain centralised training with decentralised execution.

1 Non-stationarity

IF THE OTHER AGENTS LEARN, YOUR ENVIRONMENT IS NOT AN MDP

An MDP's transition function is fixed by definition (CSCE 625 Module 10 §1). **If other agents are treated as part of the environment and those agents are learning, then P changes over time** — so the object you are solving is not an MDP at all.

So every convergence argument in this course is void. And empirically the failure is visible: **agents chase each other's changing behaviour rather than converging**, and a policy that was good last hour is exploited this hour.

Replay buffers become actively misleading rather than merely stale. A stored transition records how opponents behaved at the time, **which is no longer how they behave** — so the buffer teaches the agent to beat an opponent that no longer exists.

And the right solution concept changes. There is no 'optimal policy' independent of what the others do — the target is an equilibrium (CSCE 625 Module 12 §4), which may not be unique, may not be reachable by learning, and may be bad for everyone. **Which means even stating the goal precisely is harder here than anywhere else in the course.**

2 Self-play

SELF-PLAY GENERATES A CURRICULUM AUTOMATICALLY

Play against a copy of yourself. The opponent is always exactly at your level, so the task is always at the right difficulty — neither trivially easy nor hopeless — **which is a curriculum that nobody had to design** and is the single most valuable property of the method.

And it scales without a ceiling: as the agent improves, so does the opponent, so there is no fixed-opponent performance plateau to reach. **The task's difficulty tracks the agent's competence indefinitely.**

This is how superhuman play was reached in Go, chess, and shogi — and in AlphaZero's case with no human games at all, starting from random play, which was the genuinely striking part of the result and not merely the strength achieved.

And the mechanism is simple enough to be suspicious of. It works beautifully in two-player zero-sum games with a clear win condition, and considerably less well elsewhere — §3 explains why, and the explanation is not a limitation of implementation but of the game's structure.

```
IN ROCK-PAPER-SCISSORS, naive self-play CYCLES:
    the agent learns rock
    its opponent (a copy) learns paper
    which learns scissors
    which learns rock
```

No progress, forever -- and note that EACH STEP IS A GENUINE IMPROVEMENT against the current opponent. Nothing is malfunctioning.

THE DIAGNOSIS: the game is NON-TRANSITIVE. Strength is not a single number, so "better than the last version" does not mean "better".

THE RESPONSES:

OPPONENT POOLS -- play against a sample of PAST versions rather than only the current one. The agent must now beat everything it has previously been, which rules out cycling back to a strategy it already defeated.

FICTITIOUS SELF-PLAY -- play against the historical AVERAGE policy, which provably converges to equilibrium in two-player zero-sum games.

POPULATION METHODS (PSRO, league training) -- maintain a diverse population with distinct strategies and dedicated exploiters, which is what the StarCraft II result required.

AND THE EVALUATION PROBLEM IS REAL: "it beats the previous version" is NOT evidence of progress in a non-transitive game. Maintain a FIXED benchmark set of opponents and report against that, unchanged, throughout training.

3 Cooperation

	Competitive	Cooperative
Reward structure	Opposed — zero-sum.	Shared, or a team reward.
The hard part	Non-transitivity and cycling (§2).	Credit assignment across agents — the team won, and which agent's actions caused it?
Self-play	Works extremely well, and is the standard method.	Does not apply in the same way — playing against yourself is not cooperating with yourself.
Key difficulty	Evaluating whether progress is real.	Attributing a shared outcome to individual actions, which is the temporal credit-assignment problem in a second dimension.
Standard answer	Opponent pools, populations, leagues.	Centralised training with decentralised execution; value decomposition.

Cooperative multi-agent credit assignment is the harder open problem of the two, **and it is the one that matters for a game's allied NPCs or for a team of robots** — which is worth knowing, because the field's famous results are almost all on the competitive side and are not evidence about the cooperative one.

CENTRALISED TRAINING, DECENTRALISED EXECUTION

During training, the critic sees everything — all agents' observations and all their actions — **so from the critic's point of view the learning problem is stationary and credit assignment is tractable**, because nothing is hidden.

At execution time each agent uses only its own observation, so the deployed system is genuinely decentralised and does not require communication or a central controller.

This is the dominant architecture for cooperative multi-agent learning, and **value decomposition methods (VDN, QMIX) factor the team's value function into per-agent terms** so each agent can act greedily on its own component.

And QMIX's constraint is the clever part: require the mixing function to be monotone in each agent's value. Then maximising each agent's own value individually necessarily maximises the team value — so decentralised greedy execution is provably consistent with the centralised optimum, which is exactly the guarantee the architecture needs and is not obvious that you can get.

4 Multi-agent learning in a game engine

- **Self-play for a competitive AI opponent** in a game with a clear win condition. **The clearest fit for the method, and the sample cost is real** — budget for it, and see the closing callout.

- **Difficulty tiers taken from the opponent pool.** Checkpoints saved during training form a natural **difficulty ladder**, each one a genuine policy of a known strength — **which is a genuinely useful by-product** and a better source of difficulty levels than the handicapping of CSCE 625 Module 05 §4.
- **Emergent tactics as design input.** **Train agents and watch what they discover**, then decide deliberately whether to script it for the shipped AI, leave it for players to find, or patch it out.
- **Not for allied NPC behaviour, usually.** CSCE 625 Module 11's behaviour trees are **controllable and legible, and a learned ally is neither** — and an ally that behaves unpredictably is worse than one that behaves simply, because the player has to coordinate with it.
- **And balance testing, which is the most underrated use.** **An optimiser finds the dominant strategy before your players do** — which **uses the specification-gaming property of Module 10 as a feature rather than fighting it**, and is the application most likely to pay for itself on a real project.

THE HONEST STATUS

Two-player zero-sum with self-play is a solved success. Go, chess, shogi, heads-up poker, and several video games — **superhuman, repeatedly, by multiple groups.**

Cooperative multi-agent learning works in constrained settings — small teams, shared observations, well-specified team rewards — **and is not reliable at scale.**

General mixed-motive multi-agent learning is open, and the solution concepts themselves are contested — it is not merely that the algorithms are immature but that **what counts as success is unsettled** (CSCE 625 Module 12 §4's equilibria, which may be multiple and may be bad).

And the sample costs are enormous. The **StarCraft II and Dota 2 results used compute budgets unavailable to almost everyone** — tens of thousands of years of simulated game time. **Which is worth stating whenever those results are cited as evidence of what is possible**, because they are evidence about what is possible *at that budget*, and Module 13 makes the general version of this point.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Silver et al. — A general reinforcement learning algorithm that masters chess, shogi and Go (AlphaZero) (free)	§2's self-play result , with the curriculum property doing the work.
Vinyals et al. — Grandmaster level in StarCraft II with multi-agent reinforcement learning (free)	§2's league training , and the clearest demonstration of why opponent populations are necessary — and of the compute cost.
Rashid et al. — QMIX (free)	§3's monotone value decomposition , with the consistency argument.

Resource	What it covers for this module
Lanctot et al. — A Unified Game-Theoretic Approach to Multiagent RL (PSRO) (free)	§2's population method , and a good treatment of why non-transitivity demands it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Train two agents against each other** and plot each one's performance against a fixed benchmark. Report whether both appear to improve.
- 2 **Implement self-play on a simple game** and confirm the curriculum effect.
- 3 **Implement rock-paper-scissors self-play** and observe the cycle directly.
- 4 **Add an opponent pool** and confirm the cycling stops.
- 5 **Maintain a fixed benchmark set** and show that 'beats the previous version' and 'beats the benchmark' disagree.
- 6 **Set up a cooperative two-agent task** with a shared reward and try independent learners. Report the credit-assignment failure.
- 7 **Implement a centralised critic** and report the improvement.
- 8 **Implement value decomposition** and verify the monotonicity condition.
- 9 **Use a trained agent to balance-test a game you have**, and report the dominant strategy it found.
- 10 **Save checkpoints as difficulty tiers** and have someone play three of them. Report whether the ladder felt even.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is a multi-agent environment not an MDP?
- 2 What happens to replay buffers, and why?
- 3 What replaces 'optimal policy' as the goal?
- 4 Why does self-play produce a curriculum?
- 5 Why does naive self-play cycle, and what is the diagnosis?
- 6 Name three responses and what each forces.
- 7 Why is 'beats the previous version' insufficient?
- 8 Compare competitive and cooperative multi-agent learning on five axes.

- 9 Explain CTDE and QMIX's monotonicity constraint.
- 10 Give five uses in a game engine and the honest status of each setting.